

Progetto fine anno 1° SW

PackTrack: Sistema di Tracking Pacchi per Corriere

Classe: 1° Sviluppo Software 2025/2026

Consegna: 15 Giugno 2026, ore 13:00 via Github

Requisiti Funzionali

Un'azienda di spedizioni necessita di un sistema web per gestire le proprie spedizioni. Il sistema deve offrire:

- una pagina pubblica dove l'utente finale può tracciare il proprio pacco inserendo un codice di spedizione
- un pannello di backoffice (area admin) per gestire corrieri, pacchi e aggiornamenti di stato

Ogni spedizione deve avere un **codice di tracking univoco** generato automaticamente dal sistema al momento della creazione.

Lo storico degli stati di una spedizione deve essere **tracciato nel tempo**: il sistema non deve memorizzare solo lo stato corrente, ma l'intera sequenza di passaggi con data, ora e località, in modo da ricostruire il percorso completo del pacco.

Il **ciclo di vita di una spedizione** deve essere progettato da voi:

- definire quali stati esistono,
- quali transizioni sono ammesse tra uno stato e l'altro
- come vengono gestiti i casi problematici (consegna fallita, tentativo di riconsegna, pacco smarrito o danneggiato). Non tutti gli stati possono transitare liberamente verso qualsiasi altro stato: la logica delle transizioni deve essere coerente e documentata.

Le transizioni di stato non devono essere gestite solo a livello documentale: il sistema deve **validare attivamente** ogni cambio di stato, rifiutando transizioni non ammesse (ad esempio, un pacco "Consegnato" non può tornare "In preparazione"). La logica di validazione deve risiedere nel layer di servizio dell'applicazione.

Al momento della creazione di una nuova spedizione, il sistema deve **assegnare automaticamente un corriere** tra quelli disponibili, privilegiando il corriere con il minor numero di spedizioni attive in quel momento (bilanciamento del carico di lavoro).

Il **costo della spedizione** deve essere calcolato automaticamente dal sistema in base a criteri definiti dallo studente (ad esempio: peso del pacco, dimensioni, zona di destinazione). Lo studente deve progettare autonomamente le fasce e le regole tariffarie, documentando nel file di analisi.

Il sistema deve fornire una **stima della data di consegna**, calcolata al momento della creazione della spedizione e aggiornata ad ogni cambio di stato. I criteri di stima sono a scelta dello studente (ad esempio: zona di destinazione, tempi medi storici del corriere assegnato, tipologia di spedizione).

Lato **utente**, il sistema deve permettere di cercare una spedizione tramite il codice di tracking e visualizzarne il dettaglio completo: informazioni sul pacco, dati del mittente e del destinatario, corriere assegnato, costo della spedizione, data di consegna stimata e timeline

degli aggiornamenti di stato ordinati cronologicamente. In caso di codice non valido o inesistente, l'utente deve ricevere un feedback chiaro.

Lato **backoffice**, l'amministratore deve poter:

- visualizzare una dashboard di riepilogo con indicatori chiave (numero totale di spedizioni, spedizioni in corso, corrieri attivi);
- gestire l'anagrafica dei corrieri (visualizzazione, creazione);
- gestire le spedizioni (visualizzazione con filtri, creazione di nuove spedizioni con assegnazione a un corriere);
- aggiornare lo stato di una spedizione esistente aggiungendo un nuovo passaggio alla sua timeline.
- Export di un excel o pdf con i dati del mese (quali KPI a vostra scelta)

Il progetto deve rispettare il pattern architetturale **MVC** (Model-View-Controller) come visto a lezione, con una separazione chiara tra modelli dati, logica di routing/controller e template di presentazione.

Persistenza

Analizzare il dominio descritto nel contesto e individuare autonomamente le entità, i relativi attributi e le relazioni tra di esse.

Includere nel progetto, nella cartella *docs*:

- Diagramma ER e progettazione logica
- Bozza diagramma delle classi (Opzionale)
- Breve documento di analisi per le scelte progettuali
- DDL ed eventuali DML
- Ogni entità deve avere i campi necessari a soddisfare i requisiti funzionali descritti nel contesto

I modelli SQLAlchemy devono essere coerenti con la progettazione prodotta

Frontend

Il frontend deve essere realizzato con le tecnologie viste a lezione: HTML5 semantico, CSS (Tailwind CSS o CSS custom) e JavaScript vanilla. Il layout deve essere responsive e funzionale sia da desktop che da dispositivo mobile.

La pagina di tracking pubblica deve includere un form di ricerca e una visualizzazione a timeline dello storico stati. In caso di pacco non trovato, l'utente deve ricevere un feedback visivo chiaro.

L'area admin deve presentare una dashboard con card riassuntive, tabelle per la gestione di corrieri e spedizioni con relativi form di inserimento e aggiornamento.

Test

Il progetto deve includere almeno un file di test che verifichi la logica dell'applicazione. I test devono coprire:

- La corretta creazione delle entità e delle relazioni tra di esse
- Il rispetto dei vincoli (es. unicità del codice tracking)
- La logica di progressione degli stati di una spedizione

I test devono essere eseguibili tramite pytest o unittest.

Git e GitHub

Il repository deve contenere una cronologia di sviluppo reale e leggibile:

- minimo 10 commit significativi con messaggi descrittivi (no "fix", "update", "asd")
- file README.md con descrizione del progetto e istruzioni per eseguire l'applicazione

Criteri di valutazione

Punti aggiuntivi potranno essere assegnati a discrezione per:

- Qualità del codice: struttura dei file, leggibilità, naming coerente
- Funzionalità implementate
- Documentazione completa e leggibile
- Collezione Postman completa e funzionante
- Generazione intelligente e leggibile del codice tracking
- Validazione lato server dei dati in input
- Uso corretto di git/github
- Test coverage superiore a 50% del codice python
- Gestione errori con pagine 404/500 personalizzate
- Opzionale: mockup Figma delle pagine principali (link nel README o nella cartella docs/)
- Deploy online dell'applicazione raggiungibile via URL pubblico con link nel README

Consegna

La consegna del progetto è prevista entro **lunedì 15 giugno 2026 alle ore 13.00** via github.
Farà **fede l'orario dell'ultimo commit Git** contenente il commento esatto:

```
"VERSIONE FINALE PROGETTO FINE CORSO 1SW"
```

In caso di ritardo nella consegna, sarà applicata una **penalità di 2 punti per ogni giorno di ritardo**.